Project Report – Group 22

1 Project Overview

Project Title: Tatou-2026

Course/Module: SoftSEC

Group Number: 22

Date: 17 OCT 2026

Repository URL: https://github.com/tiagomiguel29/tatou-2026

1.1 Project Repository

Work is organized across the main branch and member branches.

- Main: shared project integration.
- Niloy-samm-security-testing: SAMM assessment and security testing.
- Branch_Kumar : Watermarking, Tatou's API robustness and security issues assessment.
- Tiago: watermarking, RMAP integration, security fixes and mutation testing.
- Niloy-phase1: Phase I RMAP and security work.

1.2 README.md Summary

The root README explains how to install dependencies, run the tests and deploy with Docker Compose. It also includes the mutation-testing reproduction steps and maps the upload-validation requirements to tests.

1.3 Artifacts Collection

Tiago's specialization evidence is in artifacts/tiago/mutation-testing/. It includes the short report, reproduction script, dependency lock file, code and test snapshots, raw mutation results, diffs, coverage and verification records.

2 Sopnil Roy Niloy's contribution

2.1 Flag Capture and Investigation

During Phase 1, another group managed to capture the flag of Group 22. As a result of notifying others about the problem, an investigation into the problem had to be conducted to identify how access to the flag was achieved and what vulnerability was used to do that.

During the investigation, a suspicious file was found in the storage directory of the Tatou. The file had a PDF header, but it consisted of a serialized Python payload. This payload was used to execute some operating system commands that aimed at getting the contents of the flag from /app/flag and sending it to the external server.

The payload was detected with static analysis. Execution of the payload has never taken place. With the help of the investigation, the way the application can be exploited to reveal some protected secret became clear.

2.2 Root Cause and Fixing of the Vulnerability

The investigation has shown that the dynamic plugin loading in Tatou is the vulnerability that allows exploitation of the application. The application allowed uploading plugin objects and deserializing them using Python serialization features, including dill. Thus, the loaded malicious serialized object could be used to execute any operating system command on the server.

Some security enhancements were made as follows:

- Unsafe deserialization elimination: The dill package and the feature of dynamic deserialization were removed.
- Disabling dynamic plugin loading: The dynamic plugin loading endpoint returns an HTTP 410 Gone code, which means dynamic plugin loading is disabled for security reasons.
- Uploading security enhancement: Safe filename handling was added to prevent malicious path components in uploaded files.
- Correcting flag initialization: According to the professor's report, there was flag_2 instead of the configured flag in the flag file. The initialization of /app/flag in the container startup process was corrected so that /app/flag is initialized with the value of the FLAG_2 environment variable. The server was rebuilt and restarted, and the correction was checked without exposing the flag.

These steps were performed to fix the unsafe deserialization vulnerability and improve file-handling and flag-initialization features.

2.3 SAMM Work and Security Assurance Evidence

In the research work done on SAMM, the security testing practice of Tatou was examined for improvement purposes. In the course of this research work, an audit of the tests contained in the server/test/ folder was carried out. Specifically, the tests associated with generation and retrieval of the watermark link were analyzed. Although the present test set included validation of retrieval, replay, distinctness of watermarked copies, and unknown identities, there was a deficiency – negative tests of nonexistence or modifications of the version link.

To resolve this deficiency, two new tests were added –

test_nonexistent_version_link_is_rejected and test_modified_version_link_is_rejected.

The tests prove that a non-existent or manipulated version link is rejected with HTTP status 404 and with the response {"error": "document not found"}. The changes were committed with the IDs 88b954d and 9c6e217. Moreover, the GitHub Actions workflow was configured to run the security tests in automatic mode.

The full local test suite returned 26 passes, 7 skips, and 48% code coverage. Skipped tests were related to RMAP because of the imghdr import dependency that has been deleted from Python 3.13. Therefore, the seven tests are not considered to have passed. Based on security tests, negative case coverage, traceability to the T.LINK_GUESS threat, coverage measurement, and CI execution, Security Testing was considered an L3 candidate.

2.4 Watermark Analysis and PDF Recipient Investigation

The study also used watermarking techniques used by Tatou to find the recipient groups of PDFs sent in the course of the assignment.

Three different watermark extraction techniques have been applied to the set of PDFs:

- TiagoInvisibleText
- MetadataHMAC
- AddAfterEOF

Using the watermark key for watermarks, recipient IDs were obtained from five PDFs. As a result, it was possible to determine the recipient groups of the documents found.

Two other PDFs could not be assigned to any group since none of the techniques returned anything but SecretNotFoundError for them. No information regarding their structure and rendered previews was available to determine their intended recipient groups. Therefore, they remained unsolved rather than being attributed to any groups based on insufficient information.

This investigation shows how extracting watermarks can help trace document distribution and identify the recipient groups of the documents sent. It should be noted

that recovering the recipient ID only proves that a recipient was specified in the watermark; it does not prove that the group received the document or opened it.

3 Tiago Oliveira’s contribution

Git identity: Tiago Miguel · tiagomiguel29

3.1 Watermarking method

I implemented TiagoInvisibleText to associate a PDF with its intended recipient without changing its visible content. It stores an encrypted, authenticated marker in the PDF catalog and as invisible text on every page. Keeping a copy on each page allows the marker to remain available when pages are copied into another document.

I added tests for recovering the secret, rejecting incorrect keys, preserving document content and ensuring the secret is not stored in plain text. Other tests remove the catalog marker, copy pages into a new PDF and extract a single page. The marker remains readable in these tested cases, although this does not prove resistance to every removal technique.

3.2 RMAP integration and testing

I corrected the RMAP payload format so the endpoints work with the official client library and its encrypted message structure. I added integration tests using temporary OpenPGP keys, a temporary PDF and an isolated database.

The tests check that a successful handshake returns the expected link, records the version and produces a PDF containing the recipient marker. They also check that repeated retrievals create distinct copies, unknown identities are rejected and replaying the second message does not create another version.

3.3 Security improvements

I fixed missing ownership checks in the watermark creation and reading routes. Document queries now match both the document ID and the authenticated user. This prevents a user from accessing another user’s watermark operations simply by changing the document ID.

I strengthened upload validation by checking the PDF header, the final marker and whether PyMuPDF recognizes the file as a PDF. Invalid uploads are removed, and files are also cleaned up if saving their database record fails.

I removed the bash-bridge watermarking method because it exposed a command-injection risk. I also stopped publishing the MariaDB port and phpMyAdmin, reducing the services exposed outside the application.

For ordinary watermarked versions, I replaced links derived from predictable filenames with random tokens. The stored filenames also use these tokens, avoiding collisions between repeated requests. I provided a rotation tool for legacy links, with a preview

mode and a database transaction so a failed update can be rolled back. It leaves the separate RMAP link format unchanged.

3.4 Phase II specialization B: Mutation testing

I chose PDF upload validation because it handles untrusted files and directly affects which documents enter the system. Passing tests alone do not show whether they would detect an incorrect implementation. Using mutmut 3.8.0 with Python 3.12, I introduced small faults and checked which ones the tests caught.

I compared the existing tests, stronger tests on the original code, and the final tests on the corrected code. The tests exercise the real upload route with temporary files and an isolated SQLite database. I expanded four existing upload tests to 18 cases.

Stage	Tests	Mutations caught	Surviving
Original code and existing tests	4	30/35	5
Original code and stronger tests	11	31/35	4
Corrected code and final tests	**18**	**46/55**	**9**

The final stage uses a different mutation set because the code changed, so its score cannot be directly compared with the earlier stages.

Original mutant 14 exposed a missing boundary test. A PDF with exactly 1019 trailing spaces caught a one-byte offset error that the original tests missed.

Investigating original mutant 23 revealed a real bug: valid PDFs with 1020, 2048 or 8192 trailing spaces were rejected because validation only examined a fixed-size tail. I changed it to scan backward in small chunks, allowing trailing whitespace while still requiring the final PDF marker. The new regression cases now pass.

I added tests rejecting extra trailing characters, catching final-code mutants 28 and 32, and strengthened checks for stored bytes, hashes, metadata and cleanup after failures.

Nine mutations survived. Seven appear to preserve acceptance for stable files; mutants 30 and 31 preserve acceptance but can increase resource use. These are reasoned assessments rather than formal proofs, and all nine remain in the score.

The recorded full suite passed 47 tests with six applicability skips. Validator line coverage was 95.5%, compared with 50% across the project. This focused experiment does not establish the security of other routes or PDF contents.

I kept the code and test snapshots, mutation diffs, raw results, coverage and validator patch in artifacts/tiago/mutation-testing/. The dependency lock file and reproduction script allow the three stages to be rerun. A separate reproduction matched all three recorded results.

4 Kumar Rakesh Singh's Contribution

4.1 Watermarking Method

Secret_in_producer.py file implements the SecretInProducer watermarking method. Its current code does not append a payload after the PDF EOF or embed the original secret in a Base64 record. Instead, it stores the secret into the PDF's /Producer metadata field.

add_watermark() method is the main method in every watermarking method, which rejects a false secret or a key that is not a nonempty string with ValueError. It writes the secret in plain text to SECRET_DIR/secret.txt; SECRET_DIR defaults to ./storage. It computes an HMAC-SHA256 of the secret using the supplied key. It opens the PDF with pikepdf, puts the HMAC in /Producer, saves the PDF, then reads and returns the PDF bytes.

read_secret() method rejects an empty or non-string key with ValueError. It reads the HMAC from the PDF's /Producer metadata and also reads the secret-derived value from SECRET_DIR/secret.txt. It recomputes the HMAC and compares it with the metadata value. It raises SecretNotFoundError if the secret file cannot be read, or InvalidKeyError if the HMAC does not match.

_build_payload(), _mac_hex(), and extract_secret_parts() implement the secret-file write, HMAC calculation, and first/last-character extraction.

4.2 Input Validation

Test_secret_in_producer.py file is used to validate the tatou project by passing different combinations of inputs. The first function test_add_watermark_rejects_invalid_secret_or_key checks whether add_watermark() validates five different combinations of parameterized cases such as verify empty or None secrets, empty or None keys, and a non-string key. The second function test_add_watermark_rejects_invalid_pdf checks whether add_watermark() properly validates an invalid PDF file by using pytest's temporary directory for the test file and secret storage.

4.3 Flask API's health and JSON parsing

Test_api.py file is used to validate the tatou project by testing the JSON parsing and authentication.

The first test_healthz_ignores_malformed_input function sends four unusual query/body combinations such as an encoded null byte, a 4,096-character query value, malformed JSON, and a nested-looking query with binary body bytes. For each response it expects HTTP 200, JSON with exactly message and db_connected, and a boolean value for db_connected. It does not require db_connected to be True.

The second test_login_rejects_malformed_input function sends seven different empty, malformed, or incomplete JSON bodies to POST /api/login. These cover an empty body, malformed JSON, empty object and array, missing email, missing password, and empty credentials. Each is expected to return HTTP 400 with a nonempty JSON error.

The third test_protected_route_rejects_invalid_auth_headers function sends six authorization variants to GET /api/list-documents such as no header, empty header, wrong scheme, missing bearer token, malformed token, and oversized token. Each is expected to return HTTP 401 with a nonempty JSON error.